LatticeGuard: Law-Level Metamorphic Regression Oracles for Access-Control Policy Evaluators

Anonymous Author(s)

***Abstract*—Access-control policy evaluators regress when authorization engines add features, optimize semantics, or refactor policy support, yet developers rarely have complete decision tables for every principal, action, and resource. They usually know laws–deny dominance, default deny, hierarchy preservation, off-slice renaming–but not a hand-written oracle for every transformed request. LatticeGuard converts these laws into executable metamorphic obligations whose applicability predicate decides whether a candidate is counted, rejected as invalid, or reported as unsupported. In a deterministic offline evaluation over Casbin and Cedar native fragments plus an OPA/Rego normalized-decision fragment whose hierarchy closure is supplied by the reference core, LatticeGuard evaluates 15840 applicable obligations with 0 primary backend failures, rejects 3240 invalid transformations, and records 360 unsupported fragments outside the denominator. The zero-primary-failure result is a release-gating baseline, not a bug-discovery claim; sensitivity is validated separately with 36960 seeded semantic-drift controls, a 13.64% positive-control kill rate, 5040 minimized replayable counterexamples, and an 81.48% pessimistic all-candidate accounting view. Law-level regression evidence becomes credible only when applicability, rejection, theory replay, minimization, deployed-test comparison, and conservative denominator accounting are first-class evidence objects.**

***Index Terms*—access control, policy evaluation, metamorphic testing, regression oracle, authorization, reproducibility**

I. Introduction

Access-control policies evolve continuously: teams add denies, restructure role hierarchies, split resource scopes, rename policy atoms, or upgrade an evaluator. Each edit can preserve, reduce, or intentionally change an authorization decision. Yet developers seldom possess a complete oracle for every principal/action/resource request after every edit. They instead reason with laws: default deny should remain deny when no matching rule becomes reachable; a matching forbid should dominate a matching permit; adding an already implied hierarchy edge should not change a decision; and an off-slice alpha-renaming should be observationally irrelevant.

The problem is not simply missing tests. It is an oracle problem in a semantic domain where invalid transformations are easy to generate. Metamorphic testing is effective when individual expected outputs are unavailable [1]–[3], and differential testing has found important implementation faults in domains without single-output oracles [4]–[7]. However, access-control policies make naive metamorphic testing dangerous. A random extension can become reachable from the request principal; a renaming can touch the queried resource; a rule permutation can cross into a priority-sensitive fragment. Counting such cases as passed metamorphic tests creates a convincing but false denominator.

LatticeGuard's thesis is that access-control laws should be compiled into *applicability-checked obligations*. An obligation is not just a before/after policy pair. It contains a source request, follow-up request, relation identifier, applicability predicate, expected decision relation, rejection condition, unsupported-fragment condition, and minimization recipe. A candidate contributes to the measured pass denominator only if its predicate establishes that the law is applicable to the concrete before/after pair. Invalid candidates remain visible as rejected evidence; unsupported language fragments remain visible as unsupported evidence; neither can inflate results.

This design targets policy *evaluators*, not policy authoring assistants. The research object is the evaluator's treatment of deny/allow composition, principal/resource/action reachability, role/resource closure, shadowing, monotonicity, dominance, and refactoring-preserving transformations. We demonstrate the method on Casbin's model/effect structure and native role matcher language [8], [9], Cedar's PARC request model with forbid-overrides-permit semantics [10], [11], and OPA/Rego only as a normalized-decision fragment over a law object [12], [13]. The counted evidence is CPU-local, offline-replayable, and frozen before claim generation, so the paper's contribution is not a bug story in one engine but a reusable law-to-obligation method.

Together, these pieces make denominator admission the object of evaluation rather than a reporting detail. The paper contributes:

- **Law-level obligation model.** We define a typed evidence object for access-control metamorphic testing with executable applicability, rejection, unsupported-fragment, invariant, and minimization components.
- **Soundness discipline.** We identify deny-aware monotonicity as a necessary condition for principal-order obligations and certify 12 relation theorems, 48 proof obligations, 74024 finite core law instantiations, and 432 independent law-kernel replay cases.
- **Evidence-chain implementation.** We build a deterministic research system that connects public/native policy material, evaluator calls, reference semantics, bounded law certificates, semantic counterexample replay, deployed-tool crosswalks, baseline interpretation, claim traceability, dependency provenance, and reproducibility checks.
- **Evaluation.** On 120 frozen sources, LatticeGuard evaluates 15840 applicable obligations over Casbin/Cedar native fragments and an OPA/Rego normalized-decision fragment with 0 primary failures, rejects 3240 invalid transformations, records 360 unsupported rows, and pro-

duces 5040 minimized positive-control witnesses.

II. Problem Setting and Scope

A policy evaluator maps a policy P and request $q = (p, a, r, c)$ to a decision. Casbin policies commonly separate request definition, policy definition, effect, and matcher clauses [8], [14], [15]. Cedar policies evaluate a principal/action/resource/context request against permit and forbid policies with forbid precedence and default deny [10], [16], [17]. These languages expose different syntax and feature sets, but many regression risks are law-shaped.

A. Motivating task

Suppose a maintainer adds an explicit deny for suspended users, refactors a viewer/editor/owner hierarchy, and then upgrades an evaluator. A conventional unit test can check one request. It cannot easily state that the refactoring preserves all request slices whose role closure is unchanged, or that a new deny should dominate an existing allow exactly when it matches the same slice. A differential test can compare two evaluators, but it does not explain whether disagreement reflects a real semantic drift, unsupported language mismatch, or an invalid test transformation. LatticeGuard therefore treats the developer task as follows: from a law ℓ, a policy slice s, and an adapter A, generate an obligation o whose applicability can be independently recomputed.

Definition 1 (Policy-slice obligation): An obligation is a tuple $\langle \ell, P, q, P', q', \pi_\ell, \rho_\ell, u_\ell, I_\ell, m_\ell \rangle$ where π_ℓ is the applicability predicate, ρ_ℓ rejects invalid transformations, u_ℓ marks unsupported fragments, I_ℓ is the expected decision invariant, and m_ℓ is the minimization recipe for failures.

This definition is intentionally stricter than ordinary mutation testing. A candidate that violates π_ℓ is not a failed obligation; it is a rejected oracle candidate. That distinction is the difference between a trustworthy regression oracle and a mutation generator that tests its own accidental edits.

B. Validity risks

The validity risks are concrete: the law may be unsound, the predicate may be label-driven rather than recomputed, the corpus may be synthetic, the adapter boundary may be too broad, the denominator may hide rejections, the counterexample path may be unexercised, or the paper may overstate a zero-failure result. LatticeGuard responds with evidence rather than reassurance. Predicates are recomputed from policy/request material; native policy material is checked before normalization; primary, rejected, unsupported, seeded, and proof-obligation rows are kept as disjoint evidence classes.

III. Law-Level Obligation Model

The core semantics abstracts an evaluator into a decision function $\mathcal{E}_A(P, q) \in \{\mathsf{ALLOW}, \mathsf{DENY}, \mathsf{ERR}\}$. The supported core contains principals, roles, resources, actions, role inheritance, allow rules, deny rules, and deny-overrides composition. Let $\mathsf{cl}_P(p)$ denote the role closure reachable from principal p in policy P, and let $\mathsf{match}_P(r, q)$ hold when rule r is reachable from the principal and matches the action/resource slice.

Definition 2 (Deny-overrides reference core): The reference decision is DENY if any matching deny exists; otherwise ALLOW if any matching allow exists; otherwise DENY. A law predicate is sound for the core when every counted candidate satisfying the predicate also satisfies its invariant under this reference decision.

The model is not a replacement for evaluator backends. It is a guardrail for the oracle catalog. Each law is first checked against the reference core, then exercised against declared backend fragments. This two-level design prevents a relation from being admitted merely because a current implementation happens to agree with itself.

Lemma 1 (Denominator integrity): A row contributes to the pass denominator iff its computed status is APPLICABLE_EVALUATED and its before/after adapter decisions satisfy the declared invariant. Rows with computed statuses REJECTED_NOT_COUNTED, UNSUPPORTED_NOT_COUNTED, ERROR_RECORDED, or TIMEOUT_RECORDED cannot be counted as passes.

The admission rule is frozen before backend outcomes are interpreted. Each relation contract names its predicate, invariant, rejection rule, and minimizer; each candidate receives a predicate-input hash and witness hash before it can reach the counted stream. The predicate is allowed to consult the reference core to decide law side conditions, but it does not read adapter outcomes or generated result ledgers. Candidates rejected by the predicate or support boundary retain rejection/support evidence but no before/after adapter outcome, so they cannot be promoted or demoted after a backend result is observed.

Lemma 2 (Deny-aware principal monotonicity): Under deny-overrides semantics, an allow decision is preserved by substituting a principal with a role-closure superset only if the follow-up slice has no newly matching deny. Without the no-new-deny side condition, the law is unsound.

The second lemma is not cosmetic. Negative PA witnesses show that a closure-superset predicate alone is insufficient: a larger closure can introduce a matching deny. The frozen PA predicate therefore requires both a closure superset and absence of follow-up deny interference.

Theorem 1 (Bounded core law certificate): For the finite core bound used in the study–three roles, two users, one action, one resource, acyclic role inheritance, representative role assignments, and rules up to the frozen bound–all 12 relation families have theorem rows whose side conditions discharge 48 executable proof obligations and satisfy their declared invariants for every counted candidate generated by their predicates.

The theorem is reported as an executable certificate rather than as an informal proof sketch only. The checker enumerates finite policies and requests, applies each relation generator and predicate, checks the invariant against the reference core, and emits a digest over relation contracts, soundness rows, theorem rows, proof-obligation rows, and the model summary. The current run checks 74024 cases with 0 failures and 48 discharged proof obligations. The bound is finite, but the

TABLE I
LAW CATALOG. EACH ROW IS MATERIALIZED AS EXECUTABLE PREDICATE, INVARIANT, REJECTION RULE, AND MINIMIZATION RECIPE.

ID	Law family	Counted invariant	Predicate rejects when
DD	Default deny	no reachable allow/deny remains deny	a new rule becomes reachable for the request
DO	Deny dominance	matching deny dominates matching allow	added deny does not match the same request slice
PA	Deny-aware principal monotonicity	allow preserved under role-closure superset and no new deny	closure is incomparable or a new deny interferes
DA	Deny antitonicity	reachable deny remains deny under substitution	deny witness is lost after substitution
IE	Irrelevant extension	unreachable extension preserves decision	extension touches principal/action/resource closure
ID	Idempotence	duplicate equivalent material preserves decision	duplicate is not semantically identical
HC	Hierarchy closure	implied edge preserves decision	inserted edge changes request-relevant closure
HR	Hierarchy refactoring	consistent child-role rewrite preserves decision	rewritten assignment does not preserve closure/decision
SR	Shadowed rule removal	removing dominated allow preserves deny	removed allow is not dominated by matching deny
RO	Rule-order invariance	unordered fragments preserve decision under permutation	priority or first/last-match semantics are present
AR	Alpha renaming	injective off-slice renaming preserves decision	renaming touches requested principal/resource/action
SM	Scope split/merge	equivalent scope partition preserves decision	matched resource/action set changes

TABLE II
PROOF-CERTIFICATE COMPONENTS.

Component	Evidence check
Predicate witness	why the law applies to this before/after pair
Theorem ledger	12 relation theorem rows and 48 proof obligations
Reference replay	whether the law is sound in the bounded core
Backend replay	whether each declared backend fragment satisfies the invariant
Minimized witness	whether irrelevant policy material was removed without changing the failing law slice

workflow matters: any future relation change must pass the same certificate before it can enter the primary denominator.

A. Proof obligations behind the laws

A relation family is admitted only after four obligations are discharged. First, the generator must expose the syntactic edit it made, such as adding a deny, splitting a scope, or replacing a role assignment. Second, the predicate must recompute semantic reachability from the concrete before/after policies; it cannot rely on a label supplied by the generator. Third, the invariant must be checked against both the reference core and the backend decision pair. Fourth, the minimizer must preserve the same predicate witness while deleting irrelevant policy material. These obligations are intentionally redundant. If a generator bug produces an invalid follow-up policy, the predicate rejects it. If a predicate is too weak, the reference checker can expose a counterexample. If a backend disagrees, the minimizer produces the smallest replayable slice that still satisfies the predicate.

This structure is why LatticeGuard treats theory and execution as one argument. The formal model rules out invalid law definitions; the adapter layer rules out vacuous formalism; the source provenance rules out hidden benchmark construction; and the claim checks rule out paper-only arithmetic. A reader can question any edge in the chain and find a corresponding evidence class rather than a prose assurance.

Require: subjects S, adapters A, relation contracts L
for all $s \in S$, $\ell \in L$ **do**
  propose candidate (P, q, P', q') for relation ℓ
  compute ρ_ℓ, u_ℓ, and π_ℓ from policy/request material
  if ρ_ℓ holds **then**
    record rejected, not counted; **continue**
  end if
  if u_ℓ holds **then**
    record unsupported, not counted; **continue**
  end if
  if not π_ℓ **then**
    record rejected, not counted; **continue**
  end if
  for all $A_i \in A$ **do**
    evaluate $d = \mathcal{E}_{A_i}(P, q)$ and $d' = \mathcal{E}_{A_i}(P', q')$
    if $I_\ell(d, d')$ **then**
      record pass with fixture hashes
    else
      minimize and write replay witness
    end if
  end for
end for

Fig. 1. Obligation generation. Predicate computation precedes adapter decisions and denominator accounting.

B. Obligation generation and minimization

LatticeGuard follows a generate–filter–evaluate–minimize pipeline. The generator proposes candidates from frozen subjects. The predicate engine recomputes applicability from the before/after policies and requests. Adapter execution happens only after applicability and unsupported-fragment checks. Failures are minimized with relation-specific reducers.

The minimizer is law-aware. A deny-dominance failure is reduced to the matching allow, matching deny, and request. A hierarchy failure keeps the edge witness and closure-relevant request. A scope split/merge failure keeps the partition

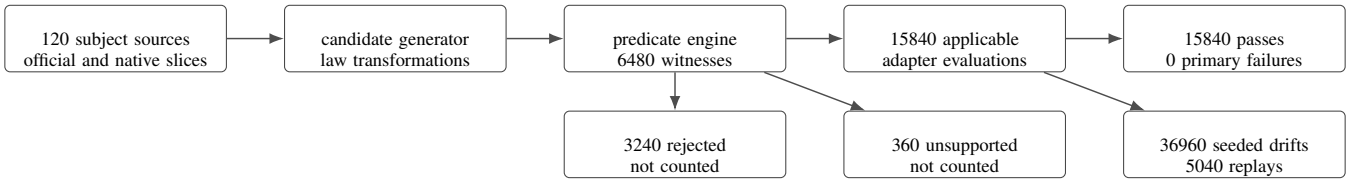

Fig. 2. Evidence flow. The contribution is the typed flow from law to predicate-checked obligation; rejected and unsupported candidates are preserved as evidence but excluded from the pass denominator.

TABLE III
EVIDENCE-CHAIN HARDENING.

Dimension	Verifier-visible evidence
Evaluator execution	2 native adapters plus 1 normalized decision harness with 15840 applicable obligations
Invalid-oracle discipline	3240 rejected candidates and 360 unsupported rows outside the pass denominator
Positive-control replay	36960 seeded rows, 5040 killed, and 5040 minimized counterexamples
Native fixture evidence	451 raw native self-tests with 0 failures
Bounded law certificate	74024 checked cases, 12 theorem rows, and 48 proof obligations
Claim binding	129 consistency checks over frozen evidence, claims, and replay traces

and matched action/resource set. This design yields small counterexamples with semantics attached, not opaque failures. It also lets a maintainer replay one failure family without rerunning the full experiment.

IV. Evidence Construction

LatticeGuard is implemented as an end-to-end research system rather than as a monolithic test generator. Its components connect law algebra, evaluator adapters, benchmark import, source linkage, proof certificates, counterexample replay, and claim checking. The implementation checks not only reported counts but also schema contracts, native fixture self-tests, predicate witnesses, counterexample families, result invariants, dependency provenance, import closure, and anonymization.

A. Adapters

The counted evidence uses unequal fragment roles. Casbin and Cedar are exercised through native evaluator semantics for the admitted slice; OPA/Rego is exercised only as a normalized-decision harness over a law object whose hierarchy closure is supplied by the reference core. The evaluated packet therefore contains 2 native adapters and 1 supplied-closure decision harness, not three interchangeable native backends.

Adapter boundary. For hierarchy stress cases, OPA/Rego receives precomputed reachable roles from LatticeGuard's trusted semantic core; Rego then executes the normalized allow/deny decision over that closure. We count OPA as a decision harness for the normalized fragment, not as evidence that recursive hierarchy closure is implemented natively in Rego. Thus hierarchy-sensitive OPA rows are normalized-fragment evidence, while Casbin and Cedar rows also exercise native hierarchy machinery in the admitted slice. Each role declares its supported fragment, decision normalization, hierarchy mapping, and material digests before entering denominators.

B. Benchmark funnel

The benchmark funnel has five explicitly separated strata. First, 6 documentation-derived slices are transcribed from official Casbin/Cedar semantics and OPA/Rego normalized-fragment semantics. Second, 7 upstream example slices are canonicalized from public project material. Third, 10 native canonical slices are imported only after raw native self-tests. Fourth, 96 semantic stress witnesses exercise relation combinations derived from the frozen law catalog and are internal hypothesis tests, not independent upstream benchmarks. Fifth, 1 generated canonical law probe remains explicitly labeled as generated. The native layer executes 451 raw self-tests with 451 passes before normalized law objects are evaluated. External-validity claims are therefore attached to the public/native strata and self-tests, while stress witnesses test law-composition coverage.

C. Native-to-canonical translation contract

Native material creates a methodological risk: a translation can accidentally simplify the policy until the benchmark becomes a generated example again. LatticeGuard therefore records three objects for every native bundle. The raw object is the original model/policy/entity material. The native self-test object is the decision table executed before normalization. The canonical object is the translated policy slice used by relation generators. The linkage check requires each canonical subject to point back to raw digests and each raw bundle to have at least one native self-test row. This design makes the benchmark funnel inspectable without requiring readers to learn every adapter's syntax before understanding the experiment.

The resulting corpus is deliberately heterogeneous. Some sources are official semantic slices, some are native fixture bundles, and one is a generated canonical law probe. LatticeGuard keeps these strata separate because collapsing them would make the evidence look larger but less meaningful. The claim we need for evaluator regression testing is not that every policy in the wild has been sampled; it is that law obligations can be generated, filtered, and replayed across public, native, and canonical material with the same denominator rules.

TABLE IV
BENCHMARK FUNNEL INVARIANTS.

Funnel stage	Required invariant
Raw native material	source digest, declared source type, license, adapter family
Native self-test	raw decision executed before normalization
Canonical subject	link to raw bundle or explicit generated label
Relation candidate	source identifier preserved through predicate and adapter rows
Paper claim	numeric macro derived from checked result evidence, not manual count

TABLE V
CURRENT EVIDENCE SUMMARY.

Metric	Value
Native adapters plus decision harnesses	2+1
Subject sources	120
Documentation-derived slices	6
Upstream example slices	7
Native canonical slices	10
Semantic stress witnesses	96
Generated law probe	1
Relation families	12
Applicable primary obligations	15840
Primary obligation passes	15840
Primary backend failures	0
Rejected invalid transformations	3240
Unsupported fragments	360
Seeded semantic-drift rows	36960
Killed seeded rows	5040
Minimized counterexamples	5040
Bounded model-check cases	74024

V. RESULTS

The evaluation asks five questions: RQ1, can executable predicates separate applicable obligations from invalid transformations? RQ2, do counted evaluator backends satisfy the law catalog over frozen subjects? RQ3, do seeded drifts exercise failure/minimization paths? RQ4, is the law catalog backed by executable theory rather than prose? RQ5, do deployed idioms and component ablations explain away the contribution?

The primary pass rate is a regression-oracle result, not a classifier score. If a current backend honors the declared law over a supported fragment, the expected primary outcome is a pass; a failure would indicate a semantic regression or wrapper/reference disagreement. We therefore validate sensitivity separately with seeded semantic drifts that intentionally break deny precedence, hierarchy closure, forbid effects, permit stripping, and allow stripping. The primary result is exactly 15840/15840 passes over pre-admitted obligations. A separate denominator-pressure view pessimistically counts rejected and unsupported candidates against the run, yielding a 81.48% accounting view; it audits the admission boundary and does not replace the primary rate. This separation lets the experiment report zero observed primary law violations without using the absence of bugs as evidence that the oracle is sensitive.

Fig. 3. Claim-verified experimental scale. Bar lengths are normalized to seeded control rows; labels show checked counts.

A. RQ1: applicability discipline

The predicate engine computed 6480 candidate witnesses. Of these, 15840 adapter-level obligations entered the denominator; 3240 invalid transformations and 360 unsupported fragments were separately reported. The most important rejected families are default-deny candidates that add reachable allows, hierarchy transformations that reverse a parent relation, alpha-renamings that touch the requested principal, scope splits that drop the only matching permission, and rule-order tests in priority-sensitive fragments. These are not failures of LatticeGuard; they are evidence that the oracle does not count unsound tests.

B. RQ2: counted backend execution

Table V reports the primary execution. Casbin and Cedar native fragments plus the OPA/Rego normalized-decision harness pass all 15840 applicable obligations in the evaluated slice, with no primary errors or timeouts. Cross-backend comparable rows have 0 discrepancies over 5280 comparable decisions. Zero discrepancies is expected on this normalized deny-overrides fragment because comparison is admitted only after support and validity filtering: 360 unsupported rows and 3240 invalid transformations remain visible but do not enter the shared-fragment differential. The result is a consistency check over the predeclared comparable subset, not evidence that the three roles have identical native semantics or complete language coverage.

C. RQ3: seeded semantic drifts

Because the current counted backends do not violate the evaluated obligations, the counterexample path is exercised with seeded semantic-drift evaluators. The drift families cover precedence violations, hierarchy-closure loss, and accidental permission loss, which are the policy-regression patterns the law catalog is designed to isolate rather than a post hoc set of easy mutants. LatticeGuard kills 5040 of 36960 seeded rows and emits 5040 minimized counterexamples. The 13.64% kill rate is intentionally lower than a program mutation score: same-stream point checks expose 13320 decision mismatches, but only 5040 rows become law-level kills after the relation predicate admits them; the remaining point-only changes stay visible outside the law-failure claim. A separate semantic replay re-evaluates every minimized witness from stored before/after request material and seeded-mutant semantics.

TABLE VI
QUALITY CHALLENGES AND EVIDENCE.

Objection	Response
Predicates are labels	Predicate engine recomputes status and witness from policy/request material
Benchmark is shallow	451 raw native fixture self-tests plus 120 frozen sources
Denominator hides failures	Rejected/unsupported statuses are disjoint from pass denominator
Theory is informal	74024 bounded cases, 432 kernel replays, and proof certificate
Counterexamples are not replayable	5040 minimized witnesses plus semantic replay evidence
Evidence is not independently checkable	Frozen theory, claim, source-linkage, hygiene, and replay ledgers agree

D. RQ4: executable theory and coverage depth

RQ4 has two parts: whether the laws have executable support and whether the subjects are nontrivial. The theory side consists of 12 relation theorems, 48 proof obligations, 74024 bounded core cases, and 432 independent law-kernel replay cases over all 12 relation families. The corpus side imports raw Casbin and Cedar material, checks native decisions, then normalizes the material into law-object subjects. Coverage is measured across adapters, relation families, subject strata, and candidate statuses; every adapter/relation cell has applicable passes.

E. RQ5: baselines, replay, and claim verification

RQ5 asks whether a simpler explanation suffices. Policy test suites, request generators, differential checks, and mutation frameworks supply useful evidence, but they do not by themselves decide whether a transformation belongs in a law-level denominator. We treat deployed-tool idioms as a nearest-neighbor comparison surface rather than as a single runnable substitute: native fixture tests, OPA-style policy-test decisions, XACML/RBAC request-generation idioms, cross-adapter differential checks, property-generation without rejection, release-pair prechecks, and the full law oracle are reported with their distinct admission rules. We then use three baseline layers: a deployed-tool crosswalk over 8 idioms; a 6-row same-stream head-to-head over the same primary and seeded transformations as the law oracle; and component ablations that remove applicability, rejection, unsupported accounting, replay, native import, and bounded law certificates. The frozen evidence spine ties those ledgers to protocol-freeze, claim-traceability, and integrity rows, and robustness replay reports zero mismatches.

The deployed comparison changes the interpretation of the zero-primary-failure result. A clean run is not presented as superiority over deployed tests; those tests remain valuable at the decision level. On the seeded stream, point tests can flag more raw decision mismatches than the law oracle admits, while every law-level seeded kill overlaps a same-stream point mismatch. The claim is therefore not that point tests are useless, but that release-gating evidence needs an additional admission rule: before a decision comparison can become a regression-oracle row, the law predicate must establish that the follow-up policy should preserve or order the original decision, and any future violation must reduce to a replayable witness.

F. Claim surface and failure policy

The evaluation has a narrow but important failure policy. A primary adapter failure is counted only when a row is applicable, supported, executable, and violates the declared invariant. A rejected candidate is a success of the oracle filter, not a pass of the evaluator. An unsupported candidate is evidence about adapter/language coverage, not evidence about correctness. A timeout or adapter error is visible in the ledger and cannot be silently dropped. These rules matter because a policy-testing paper can otherwise improve its headline number by deleting hard rows, moving invalid rows into passes, or changing the denominator after seeing results.

The reported numbers are intentionally few: applicable obligations, passes, rejections, unsupported rows, seeded rows, minimized counterexamples, native self-tests, bounded model cases, mechanized kernel cases, baseline families, and source counts. All of them are generated from checked result evidence. The supplementary material contains additional diagnostics, but the main paper avoids turning into a table dump. This mirrors the intended developer workflow: the main result is the law-to-obligation mechanism; the detailed evidence remains the trace.

G. End-to-end obligation traces

The evaluation retains concrete traces for the obligations that matter most to maintainers: deny dominance, hierarchy refactoring, off-slice alpha renaming, and native-material import. Each trace records the before/follow-up policy slice, request material, predicate witness, normalized adapter decisions, reference decisions, and any rejection reason. Table VII shows how these row-level traces answer the paper's research questions without forcing the main text to enumerate every trace.

H. Ablations and baselines

LatticeGuard is not positioned against a single end-to-end tool because the closest alternatives solve different oracle problems. OPA and Cedar suites exercise expected decisions, XACML/RBAC generators build request sets, and mutation-testing systems mutate programs or tests; none decide whether an access-control transformation is applicable before it enters a regression-oracle denominator. The deployed crosswalk records what these idioms contribute, while the same-stream head-to-head makes the comparison empirical at the decision-stream level: native and OPA-style point checks report zero failures on the clean primary stream, seeded point checks expose 13320 decision mismatches, and LatticeGuard admits 5040 of those rows as law-level kills under the relation predicates. We then compare against upstream-only, cross-adapter-only, random-without-rejection, single-relation, no-minimization, no-native-import, and no-bounded-certificate alternatives over the same frozen candidates.

TABLE VII
HOW EACH RESEARCH QUESTION IS ANSWERED BY AN EXECUTABLE EVIDENCE STREAM.

RQ	Question	Evidence stream	Current outcome
RQ1	Are invalid transformations filtered before accounting?	predicate witnesses and rejection/unsupported evidence	3240 rejected, 360 unsupported
RQ2	Do counted evaluator roles satisfy declared laws?	Casbin/Cedar native decisions plus OPA/Rego normalized-fragment decisions	15840/15840 pass
RQ3	Does the failure path work?	seeded semantic-drift evaluators and minimizers	5040 killed; 5040 minimized
RQ4	Is the law catalog more than prose?	theorems, proof obligations, bounded core, kernel replay	12 theorem rows; 48 obligations; 432 kernel cases
RQ5	Do simpler baselines explain the result?	9 baseline families, 8 deployed idioms, 6 same-stream comparisons, and claim binding	no simpler baseline supplies law admission, replay, and bounded support together

The baselines show why the design is not reducible to more tests. Point tests do not generate law obligations; cross-adapter disagreement is not an oracle without applicability; random mutation without rejection admits unsound rows; single-relation variants miss multi-causal drifts; removing native import weakens provenance; and removing the bounded checker reopens the PA monotonicity bug described in Section III.

I. Baseline design criteria

The baselines were selected to be explanatory rather than ornamental. A fair baseline must share the same subjects, adapters, and replay infrastructure; otherwise an improvement could come from easier inputs rather than a better oracle. Each baseline removes one component from the evidence chain: predicate checks, rejection ledger, native material, minimization, or bounded law support. This controlled design follows oracle-problem evaluation practice [18], [19] and mutation-testing practice [20]–[23].

The seeded semantic-drift family is the positive-control sensitivity test. It does not pretend to be a discovered vulnerability. Instead, it tests whether the method can produce a law-named, minimized counterexample when an evaluator violates deny dominance, hierarchy closure, forbid precedence, allow stripping, or permit stripping. Without this control, a zero-failure primary run would leave the counterexample path untested. With it, readers can inspect both the no-failure primary result and the live failure-reduction machinery.

This baseline structure follows the evaluation logic of strong testing papers: the comparison isolates which mechanism accounts for the improvement, rather than merely reporting that a new tool found more examples. It also clarifies the contribution boundary with XACML request generation [24], [25], policy coverage metrics [26], [27], multiple-implementation policy testing [28], model-based policy tests [29], and policy change-impact analysis [30], [31].

J. Why these baselines are adversarial

A weak evaluation would compare LatticeGuard only to no testing or to manually written examples. We instead use baselines that try to explain away the contribution. Upstream-only evidence asks whether public examples already expose seeded drifts; cross-adapter evidence asks whether disagreement alone is enough; random mutation asks whether any valid-looking edit can serve as a metamorphic oracle; and single-relation ablations ask whether one law family covers the drift space. These alternatives fail for different reasons, which is the point: access-control evaluator regressions can occur in combination algorithms, role closure, shadowing, priority handling, or normalization.

TABLE VIII
ABLATION LOGIC OVER THE SEEDED FAULT MODEL.

Ablation	Failure mode exposed
No applicability predicate	invalid transformations enter denominator
No rejection evidence	denominator cannot be inspected
No unsupported accounting	skipped fragments masquerade as clean coverage
No replayable evidence	violations cannot be independently reconstructed
No native fixtures	public evidence becomes documentation-only
No bounded/kernel checker	unsound deny-oblivious PA law can be admitted
Single relation only	misses drift families outside that law
No minimization	failures are large and hard to replay
Cross-adapter only	disagreement lacks law-level explanation

VI. DISCUSSION AND VALIDITY BOUNDARIES

The engineering shift is from point examples to regression obligations. An example says whether one request is allowed or denied; a law obligation says which transformation should preserve or order a decision and why. That distinction is what makes a regression row actionable: deny dominance, hierarchy closure, alpha-renaming, and scope split/merge each carry different side conditions and different counterexample explanations. LatticeGuard therefore treats applicability predicates as the core software-engineering object, not as bookkeeping. The denominator is decided by executable predicates, rejection rules, unsupported-fragment policy, reference agreement, and bounded semantic checks rather than by an after-the-fact spreadsheet.

The same structure makes failures interpretable without overclaiming. A primary failure requires an applicable, sup-

TABLE IX
BASELINE INTERPRETATION. THE BASELINE COLUMNS DESCRIBE WHAT A SIMPLER EXPLANATION WOULD CLAIM INSTEAD OF LATTICEGUARD AND HOW THE EVIDENCE REFUTES THAT CLAIM.

Baseline	Competing explanation	Why it is insufficient
Native/OPA point tests	Deployed tests already cover the semantics	Same-stream point checks expose decision mismatches, but law predicates decide which rows become regression-oracle failures
Cross-adapter differential	Disagreement is enough as an oracle	Disagreement lacks applicability and invalid-transformation reasoning
Release-pair precheck	Version comparison is enough	A release pair becomes evidence only after the same applicability and replay gates admit it
Random valid mutation	More policy edits will find the same drifts	Valid-looking edits can change the request slice and poison the denominator
Single relation catalog	One strong relation family is enough	Drift families distribute across deny, hierarchy, shadow, order, and scope laws
No minimization	Failure detection is enough	Large failures are hard to replay and weakly explanatory
No bounded checker	Empirical rows establish law soundness	Empirical rows alone admitted the deny-oblivious PA predicate

ported, executable row whose invariant is violated; rejected candidates and unsupported fragments remain visible but do not become passes; seeded semantic drifts are positive controls rather than discovered vulnerabilities. A maintainer can reuse the same object in release checking, policy refactoring, or language porting: freeze subjects, run the relevant law families, inspect only rows whose predicates and feature support make them comparable, and minimize any violation into replay material.

A. Outcome semantics

The outcome taxonomy is part of the method because access-control testing papers are easy to overstate. If a tool reports only passes and failures, a reader cannot tell whether a transformation was valid, whether the adapter supported the fragment, whether a failure came from a real evaluator, or whether a seeded drift was being used as a positive control. LatticeGuard therefore assigns every candidate to a mutually exclusive class before aggregation. Counted rows require a predicate witness and adapter execution; excluded rows must carry a rejection or support reason; seeded rows remain separated from primary evidence. This prevents three common mistakes: inflating the denominator with invalid edits, treating unsupported language fragments as successes, and presenting synthetic faults as discovered bugs.

This taxonomy also makes extension safer. Adding an adapter is not merely adding another executable; it must declare fragment support, decision normalization, hierarchy handling, and provenance pins before it contributes to denominators. Adding a law is not merely adding a transformation; it must define the predicate, invariant, rejection condition, unsupported-fragment behavior, and minimizer. Adding sources is not merely increasing scale; each source must retain its stratum and raw linkage. These requirements are deliberately strict because the paper's central claim is about trustworthy regression oracles, not about maximizing row counts.

B. Maintainer workflow

A maintainer uses LatticeGuard by selecting the law families relevant to an evaluator release or policy refactoring, freezing the subject corpus, and rerunning the corresponding obligations. The output is not a raw disagreement list: each counted row has a law name, predicate witness, adapter decision pair, and minimization path. This lets a team start with deny dominance, default deny, and irrelevant extension, then add hierarchy, shadowing, alpha-renaming, and scope laws as language support matures.

C. Evidence chain as method

For a regression-oracle paper, replayable evidence is not an appendix; it is part of the method. The scientific object is a counted obligation, and that object only exists after several steps agree: a source slice is imported, raw native material is self-tested when available, a canonical subject is linked to its source, a law predicate admits a candidate, adapter executions produce normalized before/after decisions, the reference semantics agrees with the adapter decisions, and any violation can be minimized into replay material. If one step is informal, the result becomes a tool report rather than an oracle discipline.

This design deliberately separates two notions that are often conflated. Verification checks whether a reported claim follows from the released evidence. Validation asks whether the claim is meaningful for evaluator maintenance. LatticeGuard needs both. Claim verification prevents stale counts and missing replay inputs from entering the release evidence. Validation comes from the law catalog, native-material linkage, alternative baselines, seeded drifts, and bounded theory support. Evidence can be reproducible but scientifically weak, or scientifically plausible but hard to reproduce; the contribution is stronger only when these dimensions are tied together.

D. Evidence-overfitting controls

A common validity risk is to grow the dataset while leaving the oracle weak. LatticeGuard uses four controls.

TABLE X
OUTCOME SEMANTICS USED BY THE REGRESSION-ORACLE EVIDENCE CHAIN.

Outcome class	Meaning	Evidence retained for inspection
Applicable pass	Law predicate holds and invariant holds	before/after adapter decisions, predicate witness, fixture hashes
Primary failure	Law predicate holds and invariant fails	violated invariant, minimized counterexample, replay request
Rejected candidate	Candidate transformation is invalid for the law	rejection reason and witness; excluded from denominator
Unsupported fragment	Adapter or language fragment lies outside declared support	unsupported reason; excluded from denominator
Seeded drift kill	Known semantic drift violates a counted law	positive-control family and minimized replay material
Adapter exclusion	Evaluator path is not safely pinned before results	pre-result exclusion reason and pinning contract

TABLE XI
EVIDENCE-CHAIN CHECKS THAT SUPPORT THE MAIN CLAIMS.

Evidence lens	Property checked before a claim is reported
Claim traceability	visible counts match generated macros and claim rows
Predicate replay	computed status and witnesses are reproducible
Source linkage	raw, native, canonical, stress, and generated strata remain distinct
Proof support	relation laws have theorem rows, bounded cases, and kernel replay
Adapter agreement	counted decisions match executable reference semantics
Counterexample replay	minimized witnesses retain decision material
Dependency closure	imports resolve from declared pinned material
Hygiene and anonymity	transient, process, and identity residue are absent

TABLE XII
CONTROLS AGAINST COMMON VALIDITY ATTACKS.

Attack	Guardrail
Grow easy rows after seeing results	frozen relation contracts and protocol amendment log
Move invalid rows into passes	executable predicate and denominator check
Hide unsupported language features	unsupported evidence with adapter support matrix
Overstate public benchmark depth	separated source strata and raw-hash linkage
Claim failure sensitivity without failures	seeded drift rows and minimization replay
Claim theory from examples	bounded core model checker, kernel replay, and proof certificate
Trust wrapper translation implicitly	adapter/reference agreement over every counted decision
Risk reference-integrity failure	citation and metadata consistency check
Perfect pass rate is overfit	outcome-independent holdout and denominator-pressure summaries

TABLE XIII
WHY A ZERO-PRIMARY-FAILURE RUN IS STILL REVIEWABLE.

Validity pressure	Evidence retained in the package
Laws are vacuous	applicable, rejected, and unsupported rows remain separate
Adapters are wrappers	counted decisions are checked against reference semantics
Invalid edits were hidden	rejection witnesses stay in the released evidence
Unsupported features became passes	unsupported rows are reported outside denominators
Failure path was inert	seeded drifts produce minimized replay witnesses
Denominator was tuned	hash-based source split and source strata are fixed before aggregation

First, each relation must provide a predicate, invariant, rejection rule, unsupported-fragment rule, and minimizer before it can add counted rows. Second, the benchmark funnel keeps documentation-derived slices, upstream examples, native imports, semantic stress witnesses, and generated probes as distinct strata. Third, seeded drifts are generated independently of the primary adapter result; they can exercise the counterexample path but cannot turn a primary failure into a pass. Fourth, the replay reports a hash-based source holdout, denominator-pressure accounting, and excluded-row checks showing rejected or unsupported rows retain no adapter outcomes.

E. Reading a zero-primary-failure run

A zero-primary-failure run is useful only if the denominator is inspectable. It could mean the evaluated fragments satisfy the declared laws, or it could reflect weak laws, filtered unsupported cases, a tuned corpus, or a dead failure path. LatticeGuard separates these readings by reporting applicable passes, rejections, unsupported fragments, seeded drifts, minimized counterexamples, model-check cases, baseline families, holdout coverage, and denominator-pressure summaries alongside the primary count. The result should therefore be read as regression assurance over declared fragments, not as "no access-control bugs exist" or "the engines are equivalent."

The current run increases evidence depth without treating scale as evidence by itself: 15840 applicable obligations, 3240 rejected transformations, 360 unsupported fragments, 36960 seeded drift rows, 5040 minimized counterexamples, 432 law-kernel replay cases, and 9 baseline families retain separate accounting roles. Seeded semantic drifts show that broken law families produce minimized witnesses, while future release-pair or backend claims must enter through the same relation and adapter contracts.

This accounting also blocks two common overfitting read-

ings. First, a large clean denominator is not treated as evidence unless the rows expose why they were admitted, rejected, or left unsupported. The reviewable object is the relation between the declared law and the admitted fragment, not a pooled success rate. Second, the positive controls are outcome-independent: seeded deny-precedence, hierarchy, forbid-effect, permit-stripping, and allow-stripping drifts are not added because a backend already failed. They ask whether the oracle path would surface a violation if the relevant law family were broken. A future evaluator upgrade can therefore improve the evidence surface only by adding a relation contract, applicability predicate, rejection rule, and replay evidence, not by enlarging the denominator after seeing the aggregate result.

VII. Related Work

LatticeGuard draws first on metamorphic testing: Chen et al.'s original formulation [32], later surveys and domain studies [1], [2], and recent logic-oriented uses [3]. It also follows differential testing traditions from compilers and system implementations [4]–[7]. Symbolic execution contributes the idea of making path conditions executable [33]–[35]. Differential generation further shows how generated cases can become stronger than anecdotal tests [36], [37]. Test generation and mutation work supply the oracle-pressure background [38]–[40]. Classical symbolic and specification-based testing explain why the law predicates must be explicit [41]–[43]. LatticeGuard's boundary is that differential or generated evidence is admitted only after a law-specific applicability predicate and rejection rule establish what should be preserved.

Access-control policy analysis supplies the semantic substrate. RBAC and ABAC define the role and attribute models that make authorization laws meaningful [44]–[47]. XACML standards and verification work motivate request generation and policy coverage [26], [30], [48]. Change-impact and policy-quality studies show why syntactic coverage is not enough [27], [31]. Multi-implementation and systematic policy-testing studies show how easy it is to confuse request breadth with semantic evidence [24], [25], [28]. LatticeGuard is narrower but deeper: it targets evaluator regression with relation-specific applicability guards, explicit rejected/unsupported accounting, executable reference semantics for backend agreement, bounded law-kernel checks, and minimized witnesses.

Modern authorization systems raise the bar for novelty. Cedar exposes a precise authorization semantics and extensive public material [10], [16], [17], [49]. Its request model, public implementation material, and ecosystem bindings make it a useful regression target [11], [50]–[52]. Recent Cedar verification and policy-repair work strengthen semantic assurance, but they address a different question from release-regression oracle construction [53]–[55]. NGAC, Zelkova-style analysis, and ProfessorX show the value of richer authorization reasoning [56]–[59]. OPA exposes practical policy evaluation and testing idioms [12], [13], [60], [61]. Casbin exposes a complementary model/effect surface for evaluator regression [8], [9], [14], [62]. Release notes give ecosystem context [15], [63], [64]. Implementation packages complete the operational view [65], [66]. Zanzibar-style systems, OpenFGA, and SpiceDB show why authorization regressions matter in service-scale deployments [67]–[69]. LatticeGuard's novelty is not a new policy language or a generic harness; it is the law-to-obligation discipline replayed across evaluator fragments with explicit boundaries under reproducibility expectations for ICSE artifacts [70]–[73].

VIII. Conclusion

The main internal validity threat is a weak law predicate; LatticeGuard mitigates it with executable witnesses, reference-soundness checks, negative candidates, bounded model checking, holdout/denominator-pressure checks, and seeded controls that exercise the failure path without being reported as real vulnerabilities. The main external boundary is fragment coverage: counted rows cover Casbin/Cedar native fragments and an OPA/Rego normalized-decision fragment over documentation-derived, upstream-example, native, stress-witness, and generated subjects. OPA hierarchy rows remain a normalized-closure result, not a native recursive-closure claim.

LatticeGuard turns access-control laws into executable, applicability-checked metamorphic obligations. A row is counted only when its law-specific predicate admits it, its invariant is checked against a declared backend fragment, and any failure can be minimized into a replayable counterexample. The broader contribution is the evidence boundary: a maintainer can tell whether a future law, backend, or policy-language extension has strengthened the regression evidence chain or only enlarged the table.

For adopters, that boundary changes how regression evidence is reviewed. A clean run is not a certificate that an authorization language is bug-free; it is a compact audit trail showing which semantic relations were exercised, which candidate rewrites were rejected, which fragments remained unsupported, and whether the counterexample path still works. LatticeGuard therefore offers a conservative practice for future evaluator releases: compare them against law-named obligations, unsupported boundaries, and minimized witnesses rather than against a collection of hand-inspected examples.

IX. Data Availability

The anonymous review artifact is provided through the submission system. It contains source code, frozen subject data for policy and evaluator cases, replay evidence, generated result tables, an open-source license under MIT terms, dependency notices and metadata, and checksums needed to reproduce the reported artifact checks. It excludes credentials, private data, private infrastructure, and author-identifying repository metadata. Reviewers can rerun checks or inspect frozen evidence without private services. The archival package will preserve the same anonymous evidence surface through Zenodo after acceptance; the version-control mirror is for navigation rather than long-term archival storage.

References

[1] S. Segura, G. Fraser, A. B. Sanchez, and A. Ruiz-Cortes, "A survey on metamorphic testing," *IEEE Transactions on Software Engineering*, vol. 42, no. 9, pp. 805–824, 2016.

[2] T. Y. Chen, F.-C. Kuo, W. Ma, W. Susilo, D. Towey, J. Voas, and Z. Q. Zhou, "Metamorphic testing for cybersecurity," *Computer*, vol. 49, no. 6, pp. 48–55, 2016.

[3] M. N. Mansur, V. Wuestholz, and M. Christakis, "Dependency-aware metamorphic testing of datalog engines," in *ISSTA*, 2023, pp. 236–247.

[4] W. M. McKeeman, "Differential testing for software," *Digital Technical Journal*, vol. 10, no. 1, pp. 100–107, 1998. [Online]. Available: https://www.hpl.hp.com/hpjournal/dtj/vol10num1/vol10num1art9.pdf

[5] X. Yang, Y. Chen, E. Eide, and J. Regehr, "Finding and understanding bugs in c compilers," in *PLDI*, 2011, pp. 283–294.

[6] V. Le, M. Afshari, and Z. Su, "Compiler validation via equivalence modulo inputs," in *PLDI*, 2014, pp. 216–226.

[7] T. Petsios, A. Tang, S. J. Stolfo, A. D. Keromytis, and S. Jana, "Nezha: Efficient domain-independent differential testing," in *IEEE Symposium on Security and Privacy*, 2017, pp. 615–632.

[8] Casbin, "Syntax for models," Online documentation, 2026. [Online]. Available: https://casbin.org/docs/syntax-for-models

[9] ——, "Policy effect," Online documentation, 2026. [Online]. Available: https://casbin.org/docs/syntax-for-models#policy-effect

[10] Cedar Policy Language, "How cedar authorization works," Online documentation, 2026. [Online]. Available: https://docs.cedarpolicy.com/auth/authorization.html

[11] ——, "Authorization requests," Online documentation, 2026. [Online]. Available: https://docs.cedarpolicy.com/auth/authorization.html#request

[12] Open Policy Agent, "Policy language," Online documentation, 2026. [Online]. Available: https://www.openpolicyagent.org/docs/policy-language

[13] ——, "Opa cli reference," Online documentation, 2026. [Online]. Available: https://www.openpolicyagent.org/docs/cli

[14] Casbin, "Rbac," Online documentation, 2026. [Online]. Available: https://casbin.org/docs/rbac

[15] ——, "Priority model," Online documentation, 2026. [Online]. Available: https://casbin.org/docs/priority-model

[16] Cedar Policy Language, "Entities," Online documentation, 2026. [Online]. Available: https://docs.cedarpolicy.com/auth/entities-syntax.html

[17] ——, "Schemas," Online documentation, 2026. [Online]. Available: https://docs.cedarpolicy.com/schema/schema.html

[18] E. T. Barr, M. Harman, P. McMinn, M. Shahbaz, and S. Yoo, "The oracle problem in software testing: A survey," *IEEE Transactions on Software Engineering*, vol. 41, no. 5, pp. 507–525, 2015.

[19] S. Anand, E. K. Burke, T. Y. Chen, J. Clark, M. B. Cohen, W. Grieskamp, M. Harman, M. J. Harrold, and P. McMinn, "An orchestrated survey of methodologies for automated software test case generation," *Journal of Systems and Software*, vol. 86, no. 8, pp. 1978–2001, 2013.

[20] P. Ammann and J. Offutt, *Introduction to Software Testing*, 2nd ed. Cambridge University Press, 2016.

[21] Y. Jia and M. Harman, "An analysis and survey of the development of mutation testing," *IEEE Transactions on Software Engineering*, vol. 37, no. 5, pp. 649–678, 2011.

[22] M. Papadakis, M. Kintis, J. Zhang, Y. Jia, Y. Le Traon, and M. Harman, "Mutation testing advances: An analysis and survey," in *Advances in Computers*. Elsevier, 2019, vol. 112, pp. 275–378.

[23] R. Just, D. Jalali, and M. D. Ernst, "Defects4j: A database of existing faults to enable controlled testing studies for java programs," in *ISSTA*, 2014, pp. 437–440.

[24] A. Bertolino, F. Lonetti, and E. Marchetti, "Systematic xacml request generation for testing purposes," in *SEAA*, 2010, pp. 3–11.

[25] A. Bertolino, S. Daoudagh, F. Lonetti, and E. Marchetti, "Automatic xacml requests generation for policy testing," in *ICST*, 2012, pp. 842–849.

[26] E. Martin, T. Xie, and T. Yu, "Defining and measuring policy coverage in testing access control policies," in *ICICS*, 2006, pp. 139–158.

[27] E. Martin, J. Hwang, T. Xie, and V. C. Hu, "Assessing quality of policy properties in verification of access control policies," in *ACSAC*, 2008, pp. 163–172.

[28] N. Li, J. Hwang, and T. Xie, "Multiple-implementation testing for xacml implementations," in *TAV-WEB*, 2008, pp. 27–33.

[29] A. Pretschner, T. Mouelhi, and Y. Le Traon, "Model-based tests for access control policies," in *ICST*, 2008, pp. 338–347.

[30] K. Fisler, S. Krishnamurthi, L. A. Meyerovich, and M. C. Tschantz, "Verification and change-impact analysis of access-control policies," in *ICSE*, 2005, pp. 196–205.

[31] E. Martin and T. Xie, "Automated test generation for access control policies via change-impact analysis," in *SESS*, 2007, pp. 5–11.

[32] T. Y. Chen, S. C. Cheung, and S.-M. Yiu, "Metamorphic testing: A new approach for generating next test cases," The Hong Kong University of Science and Technology, Tech. Rep. HKUST-CS98-01, 1998. [Online]. Available: https://www.cse.ust.hk/~scc/TR/98-01.pdf

[33] P. Godefroid, M. Y. Levin, and D. Molnar, "Automated whitebox fuzz testing," in *NDSS*, 2008. [Online]. Available: https://www.ndss-symposium.org/ndss2008/automated-whitebox-fuzz-testing/

[34] C. Cadar, D. Dunbar, and D. Engler, "Klee: Unassisted and automatic generation of high-coverage tests for complex systems programs," in *OSDI*, 2008, pp. 209–224. [Online]. Available: https://www.usenix.org/legacy/event/osdi08/tech/full_papers/cadar/cadar.pdf

[35] K. Sen, D. Marinov, and G. Agha, "Cute: A concolic unit testing engine for c," in *ESEC/FSE*, 2005, pp. 263–272.

[36] S. Person, M. B. Dwyer, S. Elbaum, and C. S. Pasareanu, "Differential symbolic execution," in *FSE*, 2008, pp. 226–237.

[37] K. Taneja and T. Xie, "Diffgen: Automated regression unit-test generation," in *ASE*, 2010, pp. 407–410.

[38] G. Fraser and A. Arcuri, "Evosuite: Automatic test suite generation for object-oriented software," in *ESEC/FSE*, 2011, pp. 416–419.

[39] G. Fraser and A. Zeller, "Mutation-driven generation of unit tests and oracles," in *ISSTA*, 2012, pp. 147–158.

[40] C. Nie and H. Leung, "A survey of combinatorial testing," *ACM Computing Surveys*, vol. 43, no. 2, pp. 11:1–11:29, 2011.

[41] L. A. Clarke, "A system to generate test data and symbolically execute programs," *IEEE Transactions on Software Engineering*, vol. 2, no. 3, pp. 215–222, 1976.

[42] J. C. King, "Symbolic execution and program testing," *Communications of the ACM*, vol. 19, no. 7, pp. 385–394, 1976.

[43] S. Khurshid and D. Marinov, "Testera: Specification-based testing of java programs using sat," *Automated Software Engineering*, vol. 11, no. 4, pp. 403–434, 2004.

[44] D. F. Ferraiolo and D. R. Kuhn, "Role-based access controls," in *National Computer Security Conference*, 1992, pp. 554–563. [Online]. Available: https://csrc.nist.gov/publications/detail/conference-paper/1992/10/13/role-based-access-controls/final

[45] R. S. Sandhu, E. J. Coyne, H. L. Feinstein, and C. E. Youman, "Role-based access control models," *Computer*, vol. 29, no. 2, pp. 38–47, 1996.

[46] V. C. Hu, D. Ferraiolo, R. Kuhn, A. Schnitzer, K. Sandlin, R. Miller, and K. Scarfone, "Guide to attribute based access control (abac) definition and considerations," NIST, Tech. Rep. SP 800-162, 2014.

[47] V. C. Hu, D. R. Kuhn, and D. Yaga, "Verification and test methods for access control policies/models," NIST, Tech. Rep. SP 800-192, 2017.

[48] OASIS XACML Technical Committee, "extensible access control markup language version 3.0 core specification," OASIS Standard, 2013. [Online]. Available: https://docs.oasis-open.org/xacml/3.0/xacml-3.0-core-spec-os-en.html

[49] Cedar Policy Language, "Policy examples," Online documentation, 2026. [Online]. Available: https://docs.cedarpolicy.com/policies/syntax-policy.html

[50] ——, "Cedar repository," GitHub repository, 2026. [Online]. Available: https://github.com/cedar-policy/cedar

[51] ——, "Cedar integration tests repository," GitHub repository, 2026. [Online]. Available: https://github.com/cedar-policy/cedar-integration-tests

[52] cedarpy Maintainers, "cedarpy project description," Python Package Index, 2026. [Online]. Available: https://pypi.org/project/cedarpy/4.8.3/

[53] J. W. Cutler, C. Disselkoen, A. Eline, S. He, K. Headley, M. Hicks, K. Hietala, E. Ioannidis, J. Kastner, A. Mamat, D. McAdams, M. McCutchen, N. Rungta, E. Torlak, and A. Wells, "Cedar: A new language for expressive, fast, safe, and analyzable authorization," *Proceedings of the ACM on Programming Languages*, vol. 8, no. OOPSLA1, pp. 670–697, 2024.

[54] C. Disselkoen, A. Eline, S. He, K. Headley, M. Hicks, K. Hietala, J. Kastner, A. Mamat, M. McCutchen, N. Rungta, B. Shah, E. Torlak, and A. Wells, "How we built cedar: A verification-guided approach," in *FSE Companion*, 2024.

[55] K. L. Wu, C. Jenkins, S. D. Stoller, and O. Chowdhury, "Automatically tightening access control policies with restricter," in *Tools and Algorithms for the Construction and Analysis of Systems*, ser. Lecture Notes in Computer Science, 2026, pp. 110–129.

[56] B. Tan, E. S. D. Davies, I. Ray, and M. A. Abdelgawad, "Safety analysis in the ngac model," in *SACMAT*, 2025, pp. 91–98.

[57] J. Backes, U. Berrueco, T. Bray, D. Brim, B. Cook, A. Gacek, R. Jhala, K. Luckow, S. McLaughlin, M. Menon, D. Peebles, U. Pugalia, N. Rungta, C. Schlesinger, A. Schodde, A. Tanuku, C. Varming, and D. Viswanathan, "Stratified abstraction of access control policies," in *CAV*, 2020, pp. 165–176.

[58] J. Backes, P. Bolignano, B. Cook, C. Dodge, A. Gacek, K. Luckow, N. Rungta, O. Tkachuk, and C. Varming, "Semantic-based automated reasoning for aws access policies using smt," in *FMCAD*, 2018, pp. 1–9.

[59] B. Weintraub, C. Liu, W. Enck, and C. Nita-Rotaru, "Professorx: Detecting silent vulnerabilities in policy engine implementations," in *SACMAT*, 2025, pp. 99–110.

[60] Open Policy Agent, "Policy testing," Online documentation, 2026. [Online]. Available: https://www.openpolicyagent.org/docs/policy-testing

[61] ——, "Policy reference," Online documentation, 2026. [Online]. Available: https://www.openpolicyagent.org/docs/policy-reference

[62] Casbin, "Abac," Online documentation, 2026. [Online]. Available: https://casbin.org/docs/abac

[63] Open Policy Agent, "Backwards compatibility," Online documentation, 2026. [Online]. Available: https://www.openpolicyagent.org/docs/policy-reference/#backwards-compatibility

[64] ——, "Opa release v1.17.1," GitHub release, 2026. [Online]. Available: https://github.com/open-policy-agent/opa/releases/tag/v1.17.1

[65] Casbin, "Pycasbin repository," GitHub repository, 2026. [Online]. Available: https://github.com/casbin/pycasbin

[66] ——, "pycasbin 2.8.0," Python Package Index, 2026. [Online]. Available: https://pypi.org/project/pycasbin/2.8.0/

[67] R. Pang, R. Caceres, M. Burrows, Z. Chen, P. Dave, N. Germer, A. Golynski, K. Graney, N. Kang, L. Kissner, J. Lansing, J. R. Lorch, M. Stapelberg, A. Whipkey, and X. Yuan, "Zanzibar: Google's consistent, global authorization system," in *USENIX ATC*, 2019, pp. 33–46. [Online]. Available: https://www.usenix.org/conference/atc19/presentation/pang

[68] OpenFGA, "Openfga documentation," Online documentation, 2026. [Online]. Available: https://openfga.dev/docs

[69] Authzed, "Spicedb documentation," Online documentation, 2026. [Online]. Available: https://authzed.com/docs/spicedb

[70] ICSE 2027, "Research track call for papers," Conference call, 2026. [Online]. Available: https://conf.researchr.org/track/icse-2027/icse-2027-research-track

[71] IEEE, "Ieee conference proceedings formatting guidelines," Online guidelines, 2026. [Online]. Available: https://www.ieee.org/conferences/publishing/templates.html

[72] A. Muttakin, S. Mondal, and C. K. Roy, "The state of open science in software engineering research: A case study of icse artifacts," in *ICSE*, 2026. [Online]. Available: https://arxiv.org/abs/2601.02066

[73] S. Ahmed, R. Ortiz, and N. U. Eisty, "Decade-long utilization patterns of icse technical papers and associated artifacts," in *SERA*, 2024, pp. 166–173. [Online]. Available: https://arxiv.org/abs/2404.05826